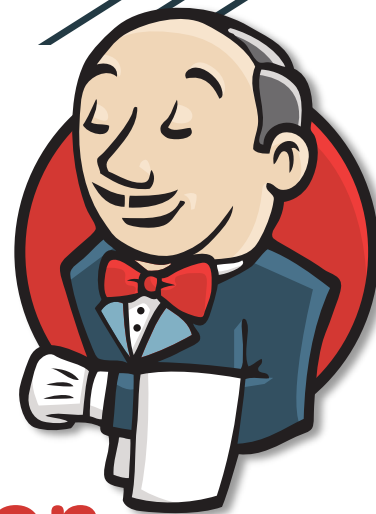




**DEVOPS**  
TOOLS ENGINEER

TOPIC-EIGHT

# Jenkins Introduction



**Arash Foroughi**

[linkedin.com/in/arashforoughi/](https://www.linkedin.com/in/arashforoughi/)

# Jenkins Introduction



# Jenkins Introduction

- Jenkins is an **open source automation** server which released at 2011 February.
- **Jenkins** is used to **build & test** your product **continuously**.
- Developers can **continuously** integrate changes into the build.
- Jenkins is the **most popular** open source CI/CD tool on the market today.
- **Automation**, including **CI/CD & test automation**, is one of the key practices that allow DevOps teams to deliver “**faster, better, cheaper**” technology solutions.
- Atlassian Bamboo, CircleCI, Eclipse Hudson, GitLab CI, JetBrains TeamCity, ... .

# Installing Jenkins on Linux



# Installing Jenkins on Linux

- On Debian & Debian-based distributions like Ubuntu Jenkins can install through apt.

```
root@master:~# curl -fsSL https://pkg.jenkins.io/debian-  
stable/jenkins.io.key | sudo tee  
/usr/share/keyrings/jenkins-keyring.asc > /dev/null  
root@master:~# echo deb [signed-by=/usr/share/keyrings/jenkins-  
keyring.asc] https://pkg.jenkins.io/debian-stable binary/ |  
sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null  
root@master:~# apt update  
root@master:~# apt install jenkins
```

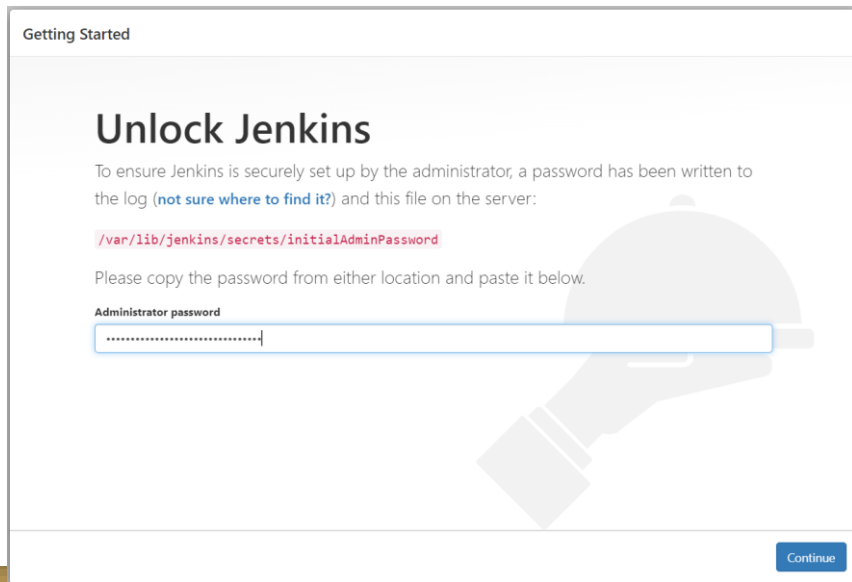
# Installing Java on Linux

- Jenkins package is going to rely on Java, so needs to install Java on your Linux.
- We will use OpenJDK which is the most popular Java version at the moment:

```
root@master:~# apt update
root@master:~# apt install openjdk-11-jdk
root@master:~# java -version
openjdk version "11.0.9.1" 2020-11-04
OpenJDK Runtime Environment (build 11.0.9.1+1-post-Debian-1deb10u2)
OpenJDK 64-Bit Server VM (build 11.0.9.1+1-post-Debian-1deb10u2, mixed mode,
sharing)
root@master:~# systemctl start Jenkins.service
root@master:~# systemctl enable Jenkins.service
```

# Post Installation Setup Wizard

- Go to <http://localhost:8080> to see the **Jenkins Setup Wizard** page.
- ❑ **First Step: Unlock Jenkins**
  - `cat /var/lib/jenkins/secrets/initialAdminPassword` to see the unlock admin password in it.



The screenshot shows the 'Getting Started' section of the Jenkins web interface. The main heading is 'Unlock Jenkins'. Below it, a paragraph explains that a password has been written to the log (with a link 'not sure where to find it?') and this file on the server: `/var/lib/jenkins/secrets/initialAdminPassword`. It then asks the user to please copy the password from either location and paste it below. There is a text input field labeled 'Administrator password' with a masked password '.....'. A 'Continue' button is located at the bottom right of the form.

Getting Started

## Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/lib/jenkins/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

# Post Installation Setup Wizard

- ❑ **Second Step:** Customize Jenkins with Plugins
  - **Install suggested plugins** - to install the recommended set of plugins, which are based on most common use cases.
  - **Select plugins to install** - to choose which set of plugins to initially install. When you first access the plugin selection page, the suggested plugins are selected by default.
- ✓ If you are not sure what plugins you need, choose Install suggested plugins. You can install (or remove) plugins later via **Manage Jenkins > Manage Plugins** page in Jenkins.

# Creating the First Administrator User

## Getting Started

### Create First Admin User

Username:

Password:

Confirm password:

Full name:

E-mail address:

Jenkins 2.319.2

[Skip and continue as admin](#) [Save and Continue](#)

# Set Your Jenkins URL

## Getting Started

### Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the `BUILD_URL` environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

# Installing Jenkins on Docker



# Installing Jenkins on Docker

- For installing Jenkins on Docker, there are needs to expose some ports:

```
root@ubuntu:~# docker run -dit -p 8080:8080 -p 50000:50000 --name jenkins-  
master -v jenkins_home:/var/jenkins_home  
jenkins/jenkins:lts-jdk11
```

- ✓ **Expose Port 8080** --> By default Jenkins run on this port.
- ✓ **Expose Port 50000** --> Jenkins Master / Slave Communication.
- ✓ **Run in Detached Mode** --> Run Jenkins container in background.
- ✓ **Bind Named Value** --> Persist data of Jenkins, if it is restarted.

# Preparing our Environment - Build Accounts



# Preparing our Environment

1. Check Jenkins user has /bin/bash shell environment or not:

```
root@master:~# cat /etc/passwd | grep jenkins
```

```
root@master:~# usermod -s /bin/bash jenkins
```

2. Set a password for Jenkins user:

```
root@master:~# passwd jenkins
```

3. Add Jenkins in the “sudo” group and allow it to run any commands anywhere:

```
root@master:~# usermod -aG sudo jenkins
```

```
root@master:~# visudo --> %jenkins All=(ALL) NOPASSWD: ALL
```

4. Add private key of Jenkins user for localhost and other Jenkins Slave Nodes:

```
jenkins@master:~$ ssh-keygen
```

```
jenkins@master:~$ ssh-copy-id jenkins@localhost
```

# Our First Jenkins Build



# Manage Jenkins

 **Jenkins**

Search 

Arash Foroughi  log out

Dashboard 

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 New View

Build Queue 

No builds in the queue.

Build Executor Status 

1 Idle

2 Idle

## Manage Jenkins

### System Configuration

**Configure System**  
Configure global settings and paths.

**Global Tool Configuration**  
Configure tools, their locations and automatic installers.

**Manage Plugins**  
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.

**Manage Nodes and Clouds**  
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

### Security

**Configure Global Security**  
Secure Jenkins; define who is allowed to access/use the system.

**Manage Credentials**  
Configure credentials

**Configure Credential Providers**  
Configure the credential providers and types

**Manage Users**

# Different Types of Jenkins Projects

## Freestyle

- ❑ Used for:
  - Simple, Single Tasks
  - Like, run tests

## Pipeline

- ❑ Used for:
  - Configure Whole Delivery Flow
  - Like, Test | Build | Package | Deploy ...
  - Just for a Single Branch

## Multibranch Pipeline

- ❑ Used for:
  - Like Pipeline
  - Just for Multibranch

Enter an item name

» This field cannot be empty, please enter a valid name

# New Items



## Freestyle project

This is the central feature of Jenkins. Jenkins will build your project, combining any SCM with any build system, and this can be even used for something other than software build.



## Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



## Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



## Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



## Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



## Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.

# Create a New Folder

**General** Health metrics Properties Pipeline Libraries

**Display Name** ?

**Description**  

For the Jenkins Introduction in Sematec DevOps Class.

[Plain text] [Preview](#)

**Health metrics**  

Health metrics...

**Properties**  
**Docker Label** ?

Save Apply

 ?

# Create a New Free Style Project in the Folder

Dashboard ▸ These are the test builds ▸ MyFirstBuild ▸

General

Source Code Management

Build Triggers

Build Environment

Build

Post-build Actions

Description

Get the log output for the system.

[Plain text] [Preview](#)

☐ Discard old builds

☐ GitHub project

☐ This build requires lockable resources

☐ This project is parameterized

☐ Throttle builds

☐ Disable this project

☐ Execute concurrent builds if necessary

?

?

?

?

?

Advanced...

Source Code Management

None

Save

Apply

?

# Change the Build Step

## Build

Add build step ▲

- Execute Windows batch command
- Execute shell
- Invoke Ant
- Invoke Gradle script
- Invoke top-level Maven targets
- Run with timeout
- Set build status to "pending" on GitHub commit

## Build

Execute shell

Command

`cat /var/log/messages`

See [the list of available environment variables](#)

# Check the Status of our Project



## These are the test builds

Folder name: MyBuilds

For the Jenkins Introduction in Sematec DevOps Class.

Add description

All



| S | W | Name ↓       | Last Success | Last Failure | Last Duration |  |
|---|---|--------------|--------------|--------------|---------------|--|
|   |   | MyFirstBuild | N/A          | N/A          | N/A           |  |

Icon: S M L

Icon legend

Atom feed for all

Atom feed for failures

Atom feed for just latest builds

# Check the Status of our Project

The screenshot shows the Jenkins project interface with several callouts:

- 1**: Points to the **Build Now** button in the left sidebar.
- 2**: Points to the **Build History** tab in the top navigation bar.
- 3**: Points to the **Console Output** link in the build details view.
- 4**: Points to the **Console Output** window, which displays the following text:

```
Started by user Arash Foroughi
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/MyBuilds/MyFirstBuild
[MyFirstBuild] $ /bin/sh -xe /tmp/jenkins16347028870413838711.sh
+ cat /var/log/messages
cat: /var/log/messages: No such file or directory
Build step 'Execute shell' marked build as failure
Finished: FAILURE
```

The main interface includes a left sidebar with options: Up, Status, Changes, Workspace, Build Now, Configure, Delete Project, Move, and Rename. The top navigation bar shows Build History, trend, and a search filter. The build details view shows a failed build (#1) with a red 'X' icon and a link to the console output.

# Reconfigure Project Again

 Up

 Status

 Changes

 Workspace

 Build Now

 **Configure**

 Delete Project

 Move

 Rename

 Build History

**Project MyFirstBuild**

Full project name: MyBuilds/MyFirstBuild  
Get the log output for the system.

 Workspace

 Recent Changes

**Permalinks**

- Last build (#1), 5 min 7 sec ago
- Last failed build (#1), 5 min 7 sec ago
- Last unsuccessful build (#1), 5 min 7 sec ago
- Last completed build (#1), 5 min 7 sec ago

 trend

^

# Reconfigure Project Again

## Console Output

Started by user **Arash Foroughi**

Running as SYSTEM

Building in workspace /var/lib/jenkins/workspace/MyBuilds/MyFirstBuild

[MyFirstBuild] \$ /bin/sh -xe /tmp/jenkins13061440602196813784.sh

+ sudo cat /var/log/syslog

sudo: a terminal is required to read the password; either use the -S option to read from standard input or configure an askpass helper

Build step 'Execute shell' marked build as failure

Finished: FAILURE

```
root@master:~# visudo
```

```
--> Change "Defaults    requiretty" to "Defaults    !requiretty"
```

```
root@master:~# vi /etc/pam.d/sudo
```

```
auth            sufficient    pam_permit.so
```

# Plugin Management & Builds



# Example-1

Execute Shell Script on Remote Host Using SSH



# 1. Adding a New Slave Node

```
root@slave:~# useradd -m -u 112 -s /bin/bash jenkins
root@slave:~# groupmod -g 118 jenkins
root@slave:~# passwd jenkins
root@slave:~# visudo
    jenkins ALL=(ALL) NOPASSWD: ALL
root@slave:~# usermod -aG sudo jenkins
root@slave:~# id jenkins
    uid=112(jenkins) gid=118(jenkins) groups=118(jenkins),27(sudo)
jenkins@slave:~# ssh-keygen
jenkins@slave:~# ssh-copy-id jenkins@localhost
jenkins@master:~# ssh-copy-id jenkins@slave
```

## 2. Adding the SSH Plugin

Dashboard

New Item

People

Build History

Manage Jenkins

My Views

Lockable Resources

New View

1

2

3

4

Manage Plugins

Add, remove, disable or enable plugins that can extend the functionality of Jenkins.

ssh

Updates Available Installed Advanced

| Install ↑                           | Name                                                                                                                 | Version | Released      |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------|---------------|
| <input checked="" type="checkbox"/> | <div>SSH</div> <div>Build Wrappers</div> <div>This plugin executes shell commands remotely using SSH protocol.</div> | 2.6.1   | 3 yr 9 mo ago |
|                                     | SSH Agent                                                                                                            |         |               |

Install without restart

Download now and install after restart

Update information obtained: 18 hr ago

Go back to the top page

(you can start using the installed plugins right away)

☐

Restart Jenkins when installation is complete and no jobs are running

## 3. Configuring SSH Plugin

### Manage Jenkins

---

#### System Configuration



**Configure System**

Configure global settings and paths.



or



**Global Tool Configuration**

Configure tools, their locations and automatic installers.



# 3. Configuring SSH Plugin

**SSH remote hosts** **1**

SSH sites

[Add](#)

SSH sites that projects will want to connect

**SSH remote hosts** **2**

SSH sites

Hostname

localhost

Port

22

Credentials

[Add](#)

**❌ Credential not selected!**

**Jenkins Credentials Provider:**

**Add Credentials** **3**

Domain

Global credentials (unrestricted)

Kind

Username with password

Scope

Global (Jenkins, nodes, items, all child items, etc)

Username

jenkins

☐ Treat username as secret

Password

.....

## 4. Use Slave Node on Other Projects

**Enter an item name** **1**

*» Required field*

**Freestyle project**  
This is the central feature of Jenkins. Jenkins will build your project used for something other than software build.

**General** **2** Source Code Management Build Triggers

**Description**

[\[Plain text\]](#) [Preview](#)

**Build** **3**

Add build step ^

Execute Windows batch command

Execute shell

**Execute shell script on remote host using ssh**

Invoke Ant

Invoke Gradle script

Invoke top-level Maven targets

Run with timeout

Set build status to

**Execute shell script on remote host using ssh** **4**

**SSH site**

**Command**

# Example-2

Pulling a Code from GitHub



# 1. Creating a New Free Style Project

Enter an item name **1**

» Required field

**Freestyle project**  
This is the central feature of Jenkins. Jenkins will build using something other than software build.

General Source Code Management **2**

Description

[Plain text] [Preview](#)

☐ Discard old builds  
☒ **GitHub project**

Project url

Display name

Source Code Management **3**

☐ None  
☒ **Git**

Repositories

Repository URL

Credentials

- none -

 Add

Add Repository

Branches to build

Branch Specifier (blank for 'any')

Add Branch

## 2. Building & Getting the Console Output

 Back to Project

 Status

 Changes

 Console Output

 View as plain text

 Edit Build Information

 Delete build '#1'

 Git Build Data

 **Console Output**

Started by user [Arash Foroughi](#)  
Running as SYSTEM  
Building in workspace /var/lib/jenkins/workspace/MyGitHubBuild  
The recommended git tool is: NONE  
No credentials specified  
Cloning the remote Git repository  
Cloning repository <https://github.com/arashforoughi/mytest>  
> git init /var/lib/jenkins/workspace/MyGitHubBuild # timeout=10  
Fetching upstream changes from <https://github.com/arashforoughi/mytest>  
> git --version # timeout=10  
> git fetch --tags --force --progress -- <https://github.com/arashforoughi/mytest> +refs/heads/\*:refs/remotes/origin/\* # timeout=10  
> git config remote.origin.url <https://github.com/arashforoughi/mytest> # timeout=10  
> git config --add remote.origin.fetch +refs/heads/\*:refs/remotes/origin/\* # timeout=10  
Avoid second fetch  
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10  
Checking out Revision f48c35deb3c6a0015d953f2643f559a785f0b04d (refs/remotes/origin/master)  
> git config core.sparsecheckout # timeout=10  
> git checkout -f f48c35deb3c6a0015d953f2643f559a785f0b04d # timeout=10  
Commit message: "Update README.md"  
First time build. Skipping changelog.  
Finished: SUCCESS

```
arash@server-3:~$ cd /var/lib/jenkins/workspace/MyGitHubBuild
arash@server-3:/var/lib/jenkins/workspace/MyGitHubBuild$ ls -l
total 12
-rw-r--r-- 1 jenkins jenkins 2325 Feb  1 15:14 index.js
-rw-r--r-- 1 jenkins jenkins  62 Feb  1 15:14 README.md
-rw-r--r-- 1 jenkins jenkins 2069 Feb  1 15:14 template.js
arash@server-3:/var/lib/jenkins/workspace/MyGitHubBuild$
```

# 3. Execute a Post-Build Step

**1**

Add Branch

Repository browser

(Auto)

Additional Behaviours

Add ▾

Build Triggers

Calculate changelog against a specific branch

**2**

Check out to a sub-directory

Check out to specific local branch

```
root@master:~# mkdir /var/www/html
```

**3**

Check out to a sub-directory

Local subdirectory for repo ?

/var/www/html

Add ▾

## Console Output

**4**

```
Started by user Arash Foroughi
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/MyGitHubBuild
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/arashforoughi/mytest
> git init /var/www/html # timeout=10
ERROR: Error cloning remote repo 'origin'
hudson.plugins.git.GitException: Could not init /var/www/html
    at org.jenkinsci.plugins.gitclient.CliGitAPIImpl$5.execute(CliGitAPIImpl.java:1042)
    at org.jenkinsci.plugins.gitclient.CliGitAPIImpl$2.execute(CliGitAPIImpl.java:797)
    at hudson.plugins.git.GitSCM.retrieveChanges(GitSCM.java:1224)
```

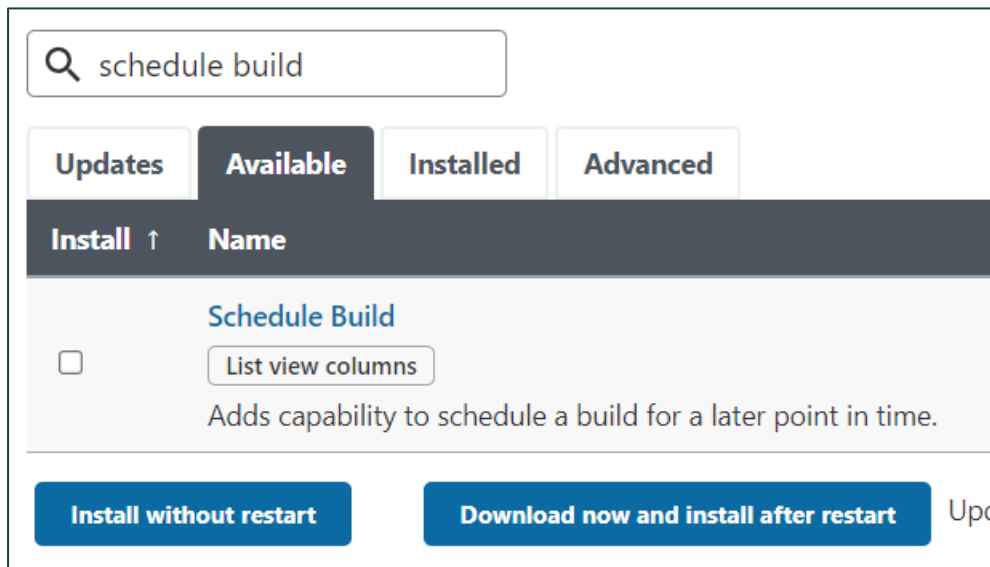
```
root@master:~# chown
jenkins:jenkins
/var/www/html
```

# Creating Scheduled Builds



# Creating Scheduled Builds

- There are two ways for scheduling builds, one way is **built-in & cron-like**, the other way is **using a plugin**.



# 1<sup>st</sup> Type: Using Schedule Build Plugin

**Enter an item name** **1**

*» Required field*

**Freestyle project**  
This is the central feature of Jenkins. Jenkins will be used for something other than software build.

**Build** **2**

**Execute shell**

Command

```
sudo cat /var/log/messages
```

See [the list of available environment variables](#)

 **Configure** **3**

 **Delete Project**

 **Move**

 **Rename**

 **Schedule Build**

**Project ScheduledBuild**

Build on:

**Schedule** **4**

**Build History** **5** [trend](#) =

 **#1**   
(pending—In the quiet period. Expires in 2 min 12 sec)

 [RSS for all](#)  [RSS for failures](#)

## 2<sup>nd</sup> Type: Using Built-in Build Triggers

### Build Triggers

- ☐ Trigger builds remotely (e.g., from scripts)
- ☐ Build after other projects are built
- ☒ Build periodically

#### Schedule

```
* / 2 * * * *
```

**⚠ Spread load evenly by using 'H/2 \* \* \* \*' rather than '\* / 2 \* \* \* \*'**

Would last have run at Wednesday, February 2, 2022 at 12:06:15 AM Iran Standard Time; would next run at Wednesday, February 2, 2022 at 12:06:15 AM Iran Standard Time.

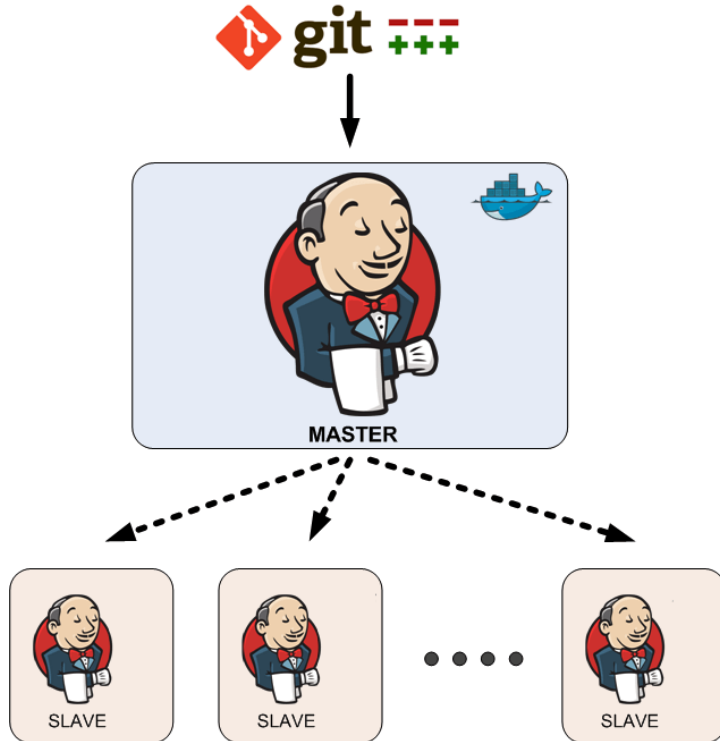
- ☐ GitHub hook trigger for GITScm polling
- ☐ Poll SCM

# Setting up a Build Slave



# Setting up a Build Slave

- Jenkins Nodes Test. Jenkins uses a **Master-Slave architecture to manage distributed builds**. In this architecture, Master and Slave nodes communicate through TCP/IP protocol. The main Jenkins server acts as the Master node that manages slaves. A slave is a Java executable that runs on a remote machine.



# Setting up a Build Slave

## Manage Jenkins 1

### System Configuration



#### Configure System

Configure global settings and paths.



#### Manage Nodes and Clouds

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

## Dashboard ▸ Nodes ▸ 2



Back to Dashboard



Manage Jenkins



New Node



Configure Clouds



Node Monitoring

## Node name 3

ubuntu-slave-1



### Permanent Agent

Adds a plain, permanent agent to this type if no other agent types

OK

# Setting up a Build Slave

|                                                                                          |   |
|------------------------------------------------------------------------------------------|---|
| Name                                                                                     | ? |
| <input type="text" value="ubuntu-slave-1"/>                                              |   |
| Description                                                                              | ? |
| <input type="text" value="Ubuntu Slave Test Node"/>                                      |   |
| Number of executors                                                                      | ? |
| <input type="text" value="4"/>                                                           |   |
| Remote root directory                                                                    | ? |
| <input type="text" value="/home/jenkins/jenkins_slave"/>                                 |   |
| Labels                                                                                   | ? |
| <input type="text"/>                                                                     |   |
| Usage                                                                                    | ? |
| <input type="text" value="Only build jobs with label expressions matching this node"/> ▼ |   |

# Setting up a Build Slave

**Launch method** ?

Launch agents via SSH ▾

**Host** ?

192.168.1.126

**Credentials** ?

jenkins/\*\*\*\*\* ▾  Add ▾

**Host Key Verification Strategy** ?

Known hosts file Verification Strategy ▾

Advanced... ?

**Availability** ?

Keep this agent online as much as possible ▾

**Node Properties** ?

☐ Disable deferred wipeout on this node

☐ Environment variables

☐ Tool Locations

Save

# Setting up a Build Slave

| S                                                                                   | Name ↓         | Architecture  | Clock Difference | Free Disk Space | Free Swap Space                                                                         | Free Temp Space | Response Time |                                                                                     |
|-------------------------------------------------------------------------------------|----------------|---------------|------------------|-----------------|-----------------------------------------------------------------------------------------|-----------------|---------------|-------------------------------------------------------------------------------------|
|    | Built-In Node  | Linux (amd64) | In sync          | 38.51 GB        |  0 B | 38.51 GB        | 0ms           |  |
|    | ubuntu-slave-1 |               | N/A              | N/A             | N/A                                                                                     | N/A             | N/A           |  |
| Data obtained                                                                       |                | 10 ms         | 8 ms             | 4 ms            | 22 min                                                                                  | 22 min          | 22 min        |                                                                                     |
|  |                |               |                  |                 |                                                                                         |                 |               |                                                                                     |



## Agent ubuntu-slave-1 (Ubuntu Slave Test Node)

This node is being launched. [See log for more details](#)

Relaunch agent

## Projects tied to ubuntu-slave-1

None



### ubuntu-slave

1 Idle

2 Idle

3 Idle

4 Idle

# Launching Jobs on the Slave Node



# Launching Jobs on the Slave Node

**Enter an item name** 1

» Required field

**Freestyle project**

This is the central feature of Jenkins. Jenkins will build your project used for something other than software build.

☐ This build requires lockable resources

☐ This project is parameterized

☐ Throttle builds

☐ Disable this project

☐ Execute concurrent builds if necessary

☒ Restrict where this project can be run

Label Expression

ubuntu-slave

**Build Environment** 3

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s)

☐ Abort the build if it's stuck

☐ Add timestamps to the Console Output

☒ Execute shell script on remote host using ssh

SSH site

Pre build script

**Build** 3

**Execute shell**

Command

See [the list of available environ](#)

**Console Output** 4

Started by user **Arash Foroughi**

Running as SYSTEM

Building remotely on **ubuntu-slave** in workspac

[MySlaveBuild] \$ /bin/sh -xe /tmp/jenkins6877

+ sudo cat /etc/hostname

server-6

Finished: SUCCESS

# Execute Commands from Slave to Master

```
jenkins@slave:~# ssh-keygen  
jenkins@slave:~# ssh-copy-id jenkins@master
```

**1**

- Then, add the new *SSH Remote Host* in **Configure System**.

Hostname

**2**

192.168.1.192

Port

22

Credentials

jenkins

 Add**Enter an item name**

MyTestRemoteBuild

» Required field

**3****Freestyle project**

This is the central feature of Jer  
used for something other than

☒ Restrict where this project can be run

Label Expression

**4**

ubuntu-slave

Label **ubuntu-slave** matches 1 node. Per

Build

**5**☒ Execute shell script on remote host using ssh

SSH site

jenkins@192.168.1.192:22

Command

sudo|cat /etc/hostname

# Create Multibranch Pipeline with Git



# Create Multibranch Pipeline with Git

Enter an item name1

» Required field

**Multibranch Pipeline**  
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Clone with SSH4

Clone with HTTPS

Open in your IDE

Visual Studio Code (SSH)

Visual Studio Code (HTTPS)

Copy URL

Build Configuration6

Mode

by Jenkinsfile

Script Path

Jenkinsfile

Display Name2

Description

Branch Sources3

Add source ^

Git

GitHub

Single repository & branch

Behaviors5

Discover branches ?

Filter by name (with regular expression)

Regular expression ?

Filter by name (with regular expression)

Regular expression ^devlmaster|feature.\*\$

Branch Source = Project you want to build

# Manage Credentials



Manage Credentials

Configure credentials

## Stores scoped to Jenkins 1

| P | Store ↓ | Domains  |
|---|---------|----------|
|   | Jenkins | (global) |

Back to credential domains

Add Credentials

2

Scope

3

Global (Jenkins, nodes, items, all child items, etc)

Global (Jenkins, nodes, items, all child items, etc)

System (Jenkins and nodes only)

## ❑ Credential Scopes: (3 Scopes)

1. **System** --> only available on Jenkins server,

Not for Jenkins jobs.

if the credential to be added is for Jenkins instance itself to interact with system administration functions, such as email authentication, agent connection, etc. Choose this option applies the scope of credential to a single object only.

2. **Global** --> Everywhere accessible.

if the credential to be added is for a Pipeline project/item. Choose this option applies the scope of credential to Pipeline project/item "object" and all its descendent objects.

# Manage Credentials



Manage Credentials

Configure credentials

Kind

Username with password

Scope

Global (Jenkins, nodes, items, all child items, etc)

Username

global

☐ Treat username as secret

Password

.....

ID

global

Description



## Global credentials (unrestricted)

Credentials that should be available irrespective of domain specification to requirements matching.

|  | ID     | Name         | Kind                   |
|--|--------|--------------|------------------------|
|  | global | global/***** | Username with password |
|  | system | system/***** | Username with password |

Icon: S M L

**ID = Reference for your credential in a  
build job or in a jenkinsfile**

# Manage Credentials



Manage Credentials  
Configure credentials

Dashboard ▸ My-Pipeline ▸ Credentials

## Stores scoped to My-Pipeline

| P | Store ↓     | Domains    |
|---|-------------|------------|
|   | My-Pipeline | (global) ▾ |

## Folder

| Domain                              |
|-------------------------------------|
| Global credentials (unrestricted) ▾ |

Kind

Username with password

Username

arashforoughi

☐ Treat username as secret

Password

.....

ID

demo-app-git-credential

Description

### ❑ Credential Scopes:

3. **Project** --> Limited to project,  
Only with multibranch  
pipeline.

## Credentials

| T | P | Store ↓ |
|---|---|---------|
|   |   | Jenkins |
|   |   | Jenkins |

Icon: S M L

## Stores scoped to Jenkins

| P | Store ↓ | Domains  |
|---|---------|----------|
|   | Jenkins | (global) |

Credentials ?

- none - ▾

- none -

arashforoughi/\*\*\*\*\*

global/\*\*\*\*\*

Discover branches ?

# Basic Jenkinsfile



# Basics of Jenkinsfile

- This is the **basic syntax** of Jenkinsfile.
- A Jenkinsfile is a text file that contains the **definition** of a Jenkins Pipeline & is checked into source control.
- It stores the **entire workflow** as **code**.
- ❑ Jenkinsfile can be written as a **Scripted Pipeline** or **Declarative Pipeline**.

```
pipeline {  
    agent any  
    stages {  
        stage("build") {  
            steps {  
            }  
        }  
    }  
}
```

# Pipeline Concepts

- Following concepts are key aspects of Jenkins Pipeline:

| Key Aspects     | Description                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pipeline</b> | A <b>Pipeline</b> is a user-defined model of a CD pipeline. It defines your entire build process, which typically includes stages for building an application, testing it and then delivering it. |
| <b>Node</b>     | A <b>node</b> is a machine which is part of Jenkins env & is capable of executing a Pipeline.                                                                                                     |
| <b>Stage</b>    | A <b>stage block</b> defines a conceptually distinct subset of tasks performed through the entire Pipeline (e.g. "Build", "Test" & "Deploy" stages).                                              |
| <b>Step</b>     | A <b>single task</b> or a step tells Jenkins what to do at a particular point in time (or "step" in the process). For example, to execute the shell command make use the sh step: sh 'make'.      |

# Pipeline Syntax Overview

| Scripted                                               | Declarative                                  |
|--------------------------------------------------------|----------------------------------------------|
| First syntax                                           | Recent addition                              |
| <a href="#">Groovy Engine</a>                          | Easier to get started with, but not powerful |
| Advanced scripting capabilities, high flexibility      | Use a pre-defined structure                  |
| Difficult to start for whom doesn't know groovy syntax | No need scripting knowledge                  |

# Declarative Pipeline

- 1) Execute this Pipeline or any of its stages, on **any available agent**.
- 2) Defines the "**Build**" stage.
- 3) Perform some steps related to the "Build" stage.
- 4) Defines the "**Test**" stage.
- 5) Perform some steps related to the "Test" stage.
- 6) Defines the "**Deploy**" stage.
- 7) Perform some steps related to the "Deploy" stage.

## Jenkinsfile (Declarative Pipeline)

```
pipeline {  
  agent any ❶  
  stages {  
    stage('Build') { ❷  
      steps {  
        // ❸  
      }  
    }  
    stage('Test') { ❹  
      steps {  
        // ❺  
      }  
    }  
    stage('Deploy') { ❻  
      steps {  
        // ❼  
      }  
    }  
  }  
}
```

# Scripted Pipeline

- 1) Execute this Pipeline or any of its stages, on **any available agent**.
- 2) Defines the "**Build**" stage. **stage blocks are optional in Scripted Pipeline syntax**.
- 3) Perform some steps related to the "Build" stage.
- 4) Defines the "**Test**" stage.
- 5) Perform some steps related to the "Test" stage.
- 6) Defines the "**Deploy**" stage.
- 7) Perform some steps related to the "Deploy" stage.

## *Jenkinsfile (Scripted Pipeline)*

```
node { ❶  
    stage('Build') { ❷  
        // ❸  
    }  
    stage('Test') { ❹  
        // ❺  
    }  
    stage('Deploy') { ❻  
        // ❼  
    }  
}
```

# A Simple Example



Scan Multibranch Pipeline Now



Scan Multibranch Pipeline Log



View as plain text

```
stage("build") {  
    steps {  
        echo 'building the application...'  
        script {  
            def test = 2 + 2 > 3 ? 'cool' : 'not cool'  
            echo test  
        }  
    }  
}  
  
stage("test") {
```

```
pipeline {  
    agent any  
    stages {  
        stage("build") {  
            steps {  
                echo 'Building Application'  
            }  
        }  
        stage("test") {  
            steps {  
                echo 'Testing Application'  
            }  
        }  
        stage("deploy") {  
            steps {  
                echo 'Deploying Application'  
            }  
        }  
    }  
}
```

# Some Important Items in Jenkinsfile



# Post Attribute in Jenkinsfile

- We defined three different stages, another thing we can do with Jenkinsfile is that at the end of it, can define “**Post**” attribute.
- What **Post** will do is basically executes some kind of scripts after all the stages done.
- They are different conditions that you can execute your scripts on it.
- **For example:** at the end of file we will say execute something always if the build failed or succeeded, like sending an email out to the team.

```
}  
}  
}  
post {  
  always {  
    //  
  }  
  success {  
    }  
}
```

```
}  
}  
}  
post {  
  always {  
    //  
  }  
  failure {  
    }  
}
```

## ❑ Conditions:

- always
- success
- failure

# Define Conditionals for Each Stage

- We can define expressions or conditionals for each state stage.
- **For example:** run tests on a specific branch.
- Here, **test steps** will be execute just if `BRANCH_NAME` variable equals to 'dev'.
- Two expressions can use with `||` or `&&`.

```
}  
stage("test") {  
  when {  
    expression {  
      BRANCH_NAME == 'dev'  
    }  
  }  
  steps {  
    echo 'testing the application...'  
  }  
}
```

# Environmental Variables

- We can create some **local variables** which it is used in the Jenkinsfile.
- There are a URL in Jenkins which contains list of all **ENV Variables**:

<http://localhost:8080/env-vars.html>

```
pipeline {
  agent any
  environment {
    NEW_VERSION = '1.3.0'
  }
  stages {
    stage("build") {
      steps {
        echo 'building the application...'
        echo "building version ${NEW_VERSION}"
      }
    }
  }
}
```

# Using Credentials in Jenkinsfile

- First, define **Credentials** in Jenkins GUI.
- Second, add a new Env Variable with value “**credentials(‘Credential\_ID’)**”
- For this feature, the plugin of “**Credentials Binding**” must be installed.

```
agent any
environment {
    NEW_VERSION = '1.3.0'
    SERVER_CREDENTIALS = credentials('')
}
stages {
```

```
}
stage("deploy") {
    steps {
        echo 'deploying the application...'
        echo "deploying with ${SERVER_CREDENTIALS}"
        sh "${SERVER_CREDENTIALS}"
    }
}
```

# Using Credentials in Jenkinsfile

- If the credential is using in just one stage, it can omitted from environment part and it can be defined under that stage like below:

```
stage("deploy") {  
    steps {  
        echo 'deploying the application...'  
        withCredentials([  
            usernamePassword(credentials: 'server-credentials', usernameVariable: USER, passwordVariable: )  
        ]) {  
            sh "some script ${USER} ${PWD}"  
        }  
    }  
}
```

# Trigger Jenkins Build Automatically



# Trigger Jenkins Build Automatically

- As we learned, we can do manually to scan branches for Jenkinsfiles and start build.
- But it's not so practical, usually we needs this whole process to work automatically.
- For example, whenever there's a change or a new commit into the git repository, we want Jenkins to be notified & the all process to be triggered & changes built.



# Ways of Triggers a Build

- **Push Notification:**
  - The Source Code Management (SCM) System detects the changes & whenever a new commit comes in it, notifies Jenkins about the changes.
- **Polling Requests:**
  - Here, Jenkins itself paused this information in regular intervals, so it basically checks itself is their new change from the repository itself or not?

1. Install Jenkins Plugin based on your Version Control System.
2. Configure Repository Server Hostname.
3. Access Token or Credential

# 1. Push Notification



# 1. Push Notification

1. Install “GitLab Web Hook” plugin in Jenkins.
2. Go to Manage Jenkins >> Configure System.
3. In GitLab go to Settings >> Integrations.
4. Use Jenkins Server URL in this page.

## Gitlab Web Hook

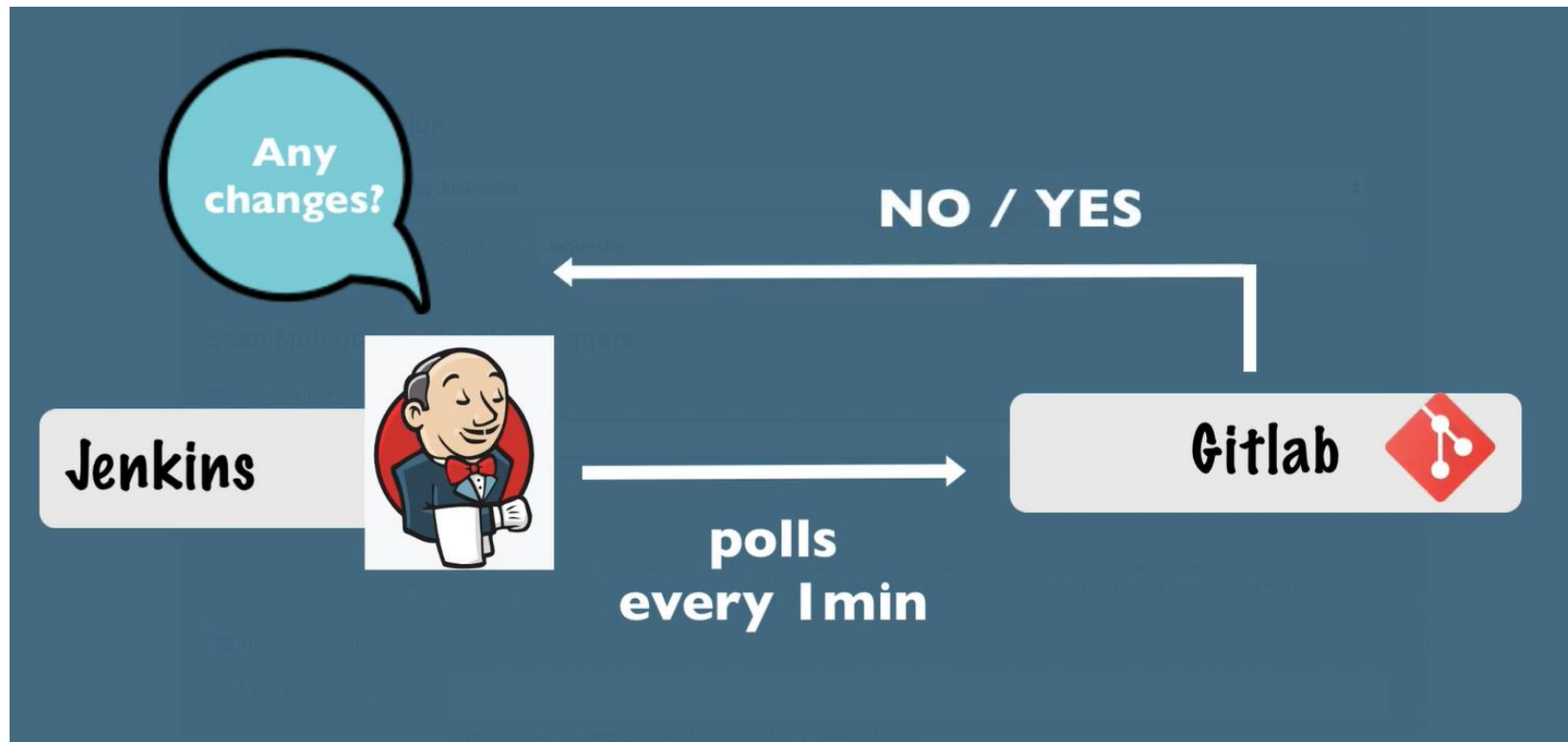
- Create new projects for merge requests ☐
- Trigger build also when pushing to merged branches ☐
- ☐ Automatic project creation

## Jenkins server URL

http://jenkins.example.com

The URL of the Jenkins server.

## 2. Polling Requests



## 2. Polling Requests

1. Go to Jenkins Job >> Configure -->
2. Configure the Interval to Jenkins must check the changes in the Gitlab.

### Scan Multibranch Pipeline Triggers

☐ Periodically if not otherwise run

### Scan Multibranch Pipeline Triggers

☒ Periodically if not otherwise run

Interval

1 minute

# Running Jenkins Pipeline to Build Docker Image from Dockerfile

<https://www.youtube.com/watch?v=MTm3cb7qiEo>



# Running Jenkins Pipeline to Build Docker Image from Dockerfile

4 lines (4 sloc) | 138 Bytes

```
1 FROM openjdk:8-jdk-alpine
2 COPY ./target/helloworld-0.0.1.war /usr/app/
3 WORKDIR /usr/app
4 ENTRYPOINT ["java", "-jar", "helloworld-0.0.1.war"]
```

## Enter an item name

» Required field



### Freestyle project

This is the central feature of Jenkins. Jenkins is used for something other than software build.



### Pipeline

Orchestrates long-running activities that can be organized into stages and/or organizing complex activities that can be organized into stages.

```
pipeline{
  agent any
  stages{
    stage('Git clone'){
      steps{
        git 'https://github.com/shazforiot/HelloWorld-Springboot-App.git'
      }
    }

    stage('maven build'){
      steps{
        sh 'mvn package'
      }
    }

    stage('Create Dockerimage'){
      steps{
        sh 'docker build -t thetips4you/springboot:latest .'
      }
    }
  }
}
```

## Running Jenkins Pipeline to Build Docker Image from Dockerfile

### Pipeline

#### Definition

Pipeline script from SCM

#### SCM

Git

#### Repositories

##### Repository URL

https://github.com/arashforoughi/HelloWorld-Springboot-App.git

##### Credentials

- none -

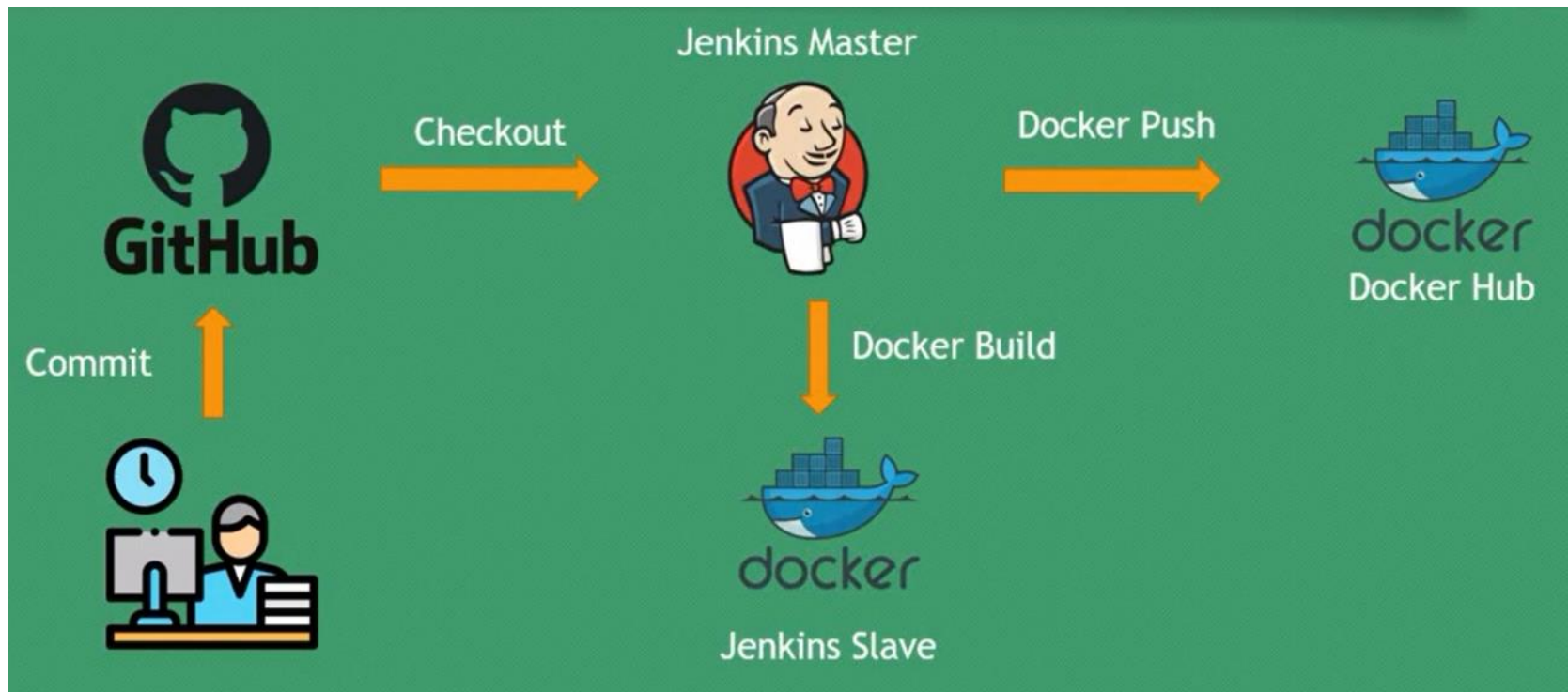
 Add

# How to Push docker image to Docker Hub using Jenkins Pipeline

<https://www.youtube.com/watch?v=8YyamgWdvFg>



# Automate Docker Build & Push to DockerHub



# Jenkins Pipeline to Build Docker Image

1. Clone to NodeJS project from GitHub
2. Build the Docker image using Docker Build
3. Login to Docker Hub
4. Push the Image to Docker Hub
5. Log Out from Docker Hub

## Pre-requisites:

- Jenkins Master
- Jenkins Slave (Docker Installed)
- Docker Hub Account
- NodeJS Sample App

[https://github.com/arashforoughi/nodeapp\\_test](https://github.com/arashforoughi/nodeapp_test)

# Create a New Access Token in DockerHub

The screenshot shows the Docker Hub account settings page. A blue header bar at the top contains the user's profile (grade, fingerprint icon, and username 9354165450). A dropdown menu is open, showing options like 'What's New', 'My Profile', 'Content Subscriptions', 'Account Settings', 'Billing', and 'Log out'. The 'Security' tab is selected in the left sidebar. The main content area shows a message about creating access tokens and a 'New Access Token' button. Below this, the 'Access Token Description' field is set to 'Jenkins-Test', and the 'Access permissions' dropdown is set to 'Read, Write, Delete'. At the bottom, a dark box displays the generated access token.

1. What's New

2. General Security

3. New Access Token

4. Access Token Description \*  
Jenkins-Test

5. To use the access token from your Docker CLI client:

1. Run `docker login -u 9354165450`
2. At the password prompt, enter the personal access token.

**c9199679-0a54-4465-b8fd-fc6ede0ac020**

# Add Docker Hub Token in Jenkins

Dashboard ▸ Credentials ▸ System ▸ Global credentials (unrestricted) **2**

 **Manage Credentials**  
Configure credentials **1**

 **DockerHub** **3** 9354165450/\*\*\*\*\*

**Enter an item name** **4**

» Required field

 **Freestyle project**  
This is the central feature of Jenkins. Jenkins

 **Pipeline**  
Orchestrates long-running activities that can

**Pipeline** **5**

Definition

Pipeline script

Script

```
1 pipeline {  
2   agent {label 'Linux'}  
3  
4   environment {  
5     DOCKERHUB_CREDENTIALS=credentials('DockerHub')  
6   }  
7  
8   stages {  
9  
10    stage ('gitclone') {  
11      steps {  
12        git 'https://github.com/arashforoughi/nodeapp_test'  
13      }  
14    }  
15  
16    stage ('Build') {  
17      steps {  
18
```

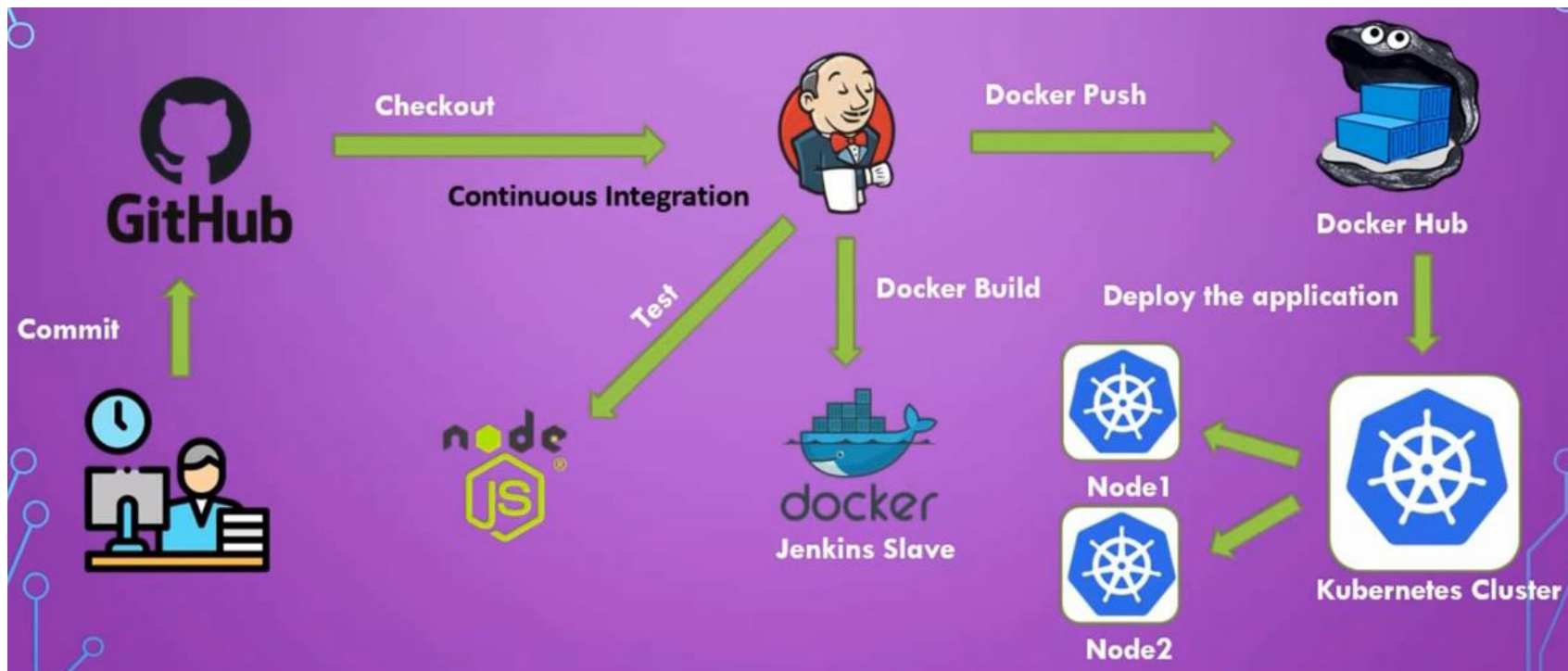
☒ Use Groovy Sandbox

# CICD Pipeline To Deploy To Kubernetes Cluster Using Jenkins | Jenkins Kubernetes Integration

[https://www.youtube.com/watch?v=XE\\_mAhxZpwU](https://www.youtube.com/watch?v=XE_mAhxZpwU)



# CI/CD Pipeline for K8s Deployment by Jenkins



# Jenkins Integration with Kubernetes

1. Docker Pipeline Plugin
2. Install Kubernetes Continuous Plugin
3. Configure Kubernetes Plugin Using Kube Config File
4. Write Jenkins Pipeline
5. Build Docker Image
6. Push Image to Docker-Hub
7. Pull the Image from Docker-Hub and Deploy to Kubernetes Cluster

## Pre-requisites:

- Your app to be deployed on K8s.
- Docker File
- Kubernetes Cluster
- Kubernetes Deployment & Service Yaml File

# Create a New Access Token in DockerHub

1

Search: docker pipeline

Updates Available Installed Advanced

Install ↑ Name

☐ Docker Pipeline

Deployment DevOps docker pipeline

Build and use Docker containers from pipelines.

Install without restart Download now and install after restart

2

Search: kubernetes continuous deploy

Updates Available Installed Advanced

Install ↑ Name

☐ Kubernetes Continuous Deploy

kubernetes

A Jenkins plugin to deploy configurations to Kubernetes cluster.

This plugin is up for adoption! We are looking for new maintainers.

Install without restart Download now and install after restart Update

3 DockerHub

Jenkins (global)

Kind

Kubernetes configuration (kubeconfig) 4

Scope

Global (Jenkins, nodes, items, all child items, etc)

ID

kubernetes

Description

kubernetes

Kubeconfig

☒ Enter directly

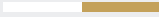
Content

cat ~/.kube/config

# CICD Pipeline To Deploy To Kubernetes Cluster Using Jenkins | Jenkins Kubernetes Integration

<https://www.youtube.com/watch?v=X7bg3JVSGlw>





Thank you.

